



УНИВЕРЗИТЕТ У НОВОМ САДУ ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА



УНИВЕРЗИТЕТ У НОВОМ САДУ

ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА

НОВИ САД

Департман за рачунарство и аутоматику

Одсек за рачунарску технику и рачунарске комуникације

ПРОЈЕКТНИ РАД

Кандидат: Милош Тришић

Број индекса: RA 39/2023

Предмет: Основи паралелног програмирања и софтверски алати

Тема рада: МАН преводилац

Ментор рада: асистент Теодора Новковић

Нови Сад, јун, 2026

Садржај

1. Увод	3
1.1 МАВН преводацац (МИПС Асемблер Високог Нивоа).....	3
1.2 Задатак пројекта	3
2. Анализа проблема	4
2.1 Превођење МАВН програма	4
2.2 Анализа животног века променљивих	4
2.3 Граф сметњи и алокација регистара	5
2.4 Генерисање кода	6
3. Концепт решења	7
3.1 Ток обраде програма	7
3.2 Архитектура система	8
3.3 Међурепрезентација (IR)	9
3.4 Процес превођења	10
4. Имплементација решења	10
4.1 Главни програм.....	10
4.2 Модул лексичке анализе.....	11
4.2.1 Коначни аутомат	11
4.2.2 Токени	11
4.2.3 Метода „Do()“	11
4.3 Модул синтаксне анализе.....	11
4.3.1 Граматика МАВН језика.....	11
4.3.2 Реализација парсера	12
4.3.3 Формирање међурепрезентације.....	12
4.4 Модул анализе животног века	13

4.4.1 Одређивање use и def скупова.....	13
4.4.2 Формирање графа тока управљања	13
4.4.3 Рачунање in и out скупова	13
4.5 Модул графа сметњи и алокације регистара	13
4.5.1 Иницијализација и формирање графа сметњи	14
4.5.2 Упрошћавање.....	14
4.5.3 Избор боја (регистара)	14
4.6 Модул генерисања кода.....	14
4.6.1 Превођење регистара и инструкција	15
4.6.2 Формирање секција и излазне датотеке	15
4.7 Додатне инструкције.....	15
4.7.1 Инструкција and	15
4.7.2 Инструкција og	16
4.7.3 Инструкција beq	16
5. Тестирање и верификација решења.....	16
5.1 Тест simple.mavn.....	16
5.2 Тест multiply.mavn.....	17
5.3 Тест extra.mavn	19
5.4 Тест loop.mavn	20
6. Закључак.....	21

1. Увод

1.1 МАВН преводацац (МИПС Асемблер Високог Нивоа)

Основни циљ сваког компајлера јесте да исправно изврши задатак превођења програма написаног на одређеном (почетном) језику на други (циљни) језик и притом као резултат превођења пружи кориснику (у нашем случају самом програмеру) излазни код у виду фајла написаног на циљном језику. У овом пројектном задатку реализован је МАВН преводацац који преводи програме написане на вишем МИПС 32-bit асемблерском језику на основни МИПС 32-bit асемблерски језик.

МАВН представља поједностављену варијанту МИПС асемблера која уводи појам регистарске променљиве. За разлику од основног МИПС-а, где програмер директно рукује ограниченим бројем физичких регистара, у МАВН програму користимо регистарске променљиве облика r1, r2, r3... Улога преводиоца је такође да ефикасно распореди ове логичке променљиве на свега 4 физичка регистра t0, t1, t2 и t3 по услову из спецификације задатка. Ово постижемо у фазама анализе животног века променљивих, бојења графа и алокације физичких регистара.

1.2 Задатак пројекта

Задатак пројекта обухвата реализацију програма који успешно врши комплетан поступак превођења. Преводацац спроводи лексичку и синтаксну анализу при чему открива и пријављује лексичке и синтаксне грешке у случају да постоје. Након тога следи фаза анализе животног века променљивих, формирање графа сметњи и фаза алокације регистара уз поменуто ограничење од највише 4 искоришћена регистра у једном тренутку. У последњој фази генерише се исправан МИПС код, који се уписује у излазну датотеку *output.s*.

Поред основних инструкција дефинисаних у поставци задатка (add, addi, sub, la, li, lw, sw, b, bltz, nor), имплементиране су три додатне асемблерске инструкције : and, or и beq. Инструкције and и or су добро познате инструкције из аритметичко-логичке групе

(коњункција односно логичко „И“ и дисјункција тј. логичко „ИЛИ“), а инструкција beq представља условни скок. За подсећање погледати Сliku 1.

AND			OR		
					
INPUT		OUTPUT	INPUT		OUTPUT
A	B	A AND B	A	B	A OR B
0	0	0	0	0	0
0	1	0	0	1	1
1	0	0	1	0	1
1	1	1	1	1	1

Слика 1 – таблица истинитости операција and и or

2. Анализа проблема

2.1 Превођење МАВН програма

Улазни МАВН програм састоји се од декларација меморијских променљивих, декларација регистарских променљивих, декларације функције, лабела и инструкција. Преводаилац најпре мора да провери да ли је тако записан програм исправан (лексички а затим и синтаксно), и тек након тога пређе у наредне фазе које треба да дају одговоре на следећа питања :

1. Када је која променљива у програму заправо потребна (жива), а када се не користи?
2. Које променљиве су истовремено живе? – не могу делити исти физички регистар
3. Које доступне физичке регистре треба да доделимо логичким променљивима?
4. Како доделити физичке регистре на довољно ефикасан начин?

2.2 Анализа животног века променљивих

Да би се логичке променљиве правилно распоредиле по регистрима, потребно је знати у ком делу програма је одређена променљива у употреби (жива). За променљиву кажемо да је жива у некој тачки тока програма уколико се њена вредност користи у некој од наредних

инструкција. Кључни закључак је следећи : Ако две променљиве **нису** истовремено живе, једино тада **могу** делити исти физички регистар, чиме директно смањујемо број потребних регистара. За сваки чвор n (инструкцију) рачунају се 4 скупа променљивих :

- $use[n]$ – скуп променљивих које чвор користи (исто што и *source*)
- $def[n]$ – скуп променљивих које чвор дефинише (исто што и *destination*)
- $in[n]$ – скуп променљивих живих на улазу у чвор n , тј. оних које морају бити доступне пре извршавања инструкција
- $out[n]$ – скуп променљивих живих на излазу из чвора n , тј. оних које ће бити потребне у некој од наредних инструкција

Скупови $use[n]$ и $def[n]$ одређују се директно из саме инструкције, док се $in[n]$ и $out[n]$ рачунају итеративно на основу следећих једначина :

$$out[n] = U in[s] \text{ за сваког следбеника } s \text{ чвора } n$$

$$in[n] = use[n] U (out[n] - def[n])$$

Анализа се спроводи уназад (од последње инструкције ка првој), јер на тај начин имамо мање итерација у решењу, што чини овај приступ ефикаснијим, а такође одмах имамо информацију које ће вредности бити потребне „у будућности“. Итерације се понављају све док се скупови in и out у 2 узастопне итерације не изједначе.

Дакле, као што смо рекли, две променљиве које се истовремено налазе у истом in или out скупу не могу делити исти физички регистар.

2.3 Граф сметњи и алокација регистара

На основу резултата анализе животног века формира се граф сметњи – неусмерен граф у коме чворови представљају регистарске променљиве, а ивица између два чвора означава да су те променљиве у сметњи тј. да им се животно век преклапа, и не можемо им

доделити исти физички регистар. У компајлерском програму, граф сметњи представљамо матрицом сметњи, чији су редови и колоне све регистарске променљиве, а вредност на пресеку означава да ли сметња постоји (1) или не (0). Матрица се попуњава на основу out скупова добијених анализом животног века.

Алокација регистара своди се на проблем бојења графа сметњи, где једна боја одговара једном физичком регистру, а повезани чворови (променљиве у сметњи) не смеју добити исту боју. Циљ је обојити граф са највише K боја (у нашем случају $K = 4$).

Алгоритам доделе ресурса састоји се од 4 подфазе :

1. Формирање (build) – попуњавање матрице сметњи.
2. Упрошћавање (simplify) - из графа се поступно уклањају чворови чији је број суседа мањи од K и постављају на стек. Чвор са мање од K суседа увек се може обојити. Уклањањем чвора отвара се могућност суседним чворовима за даље упрошћавање итеративно.
3. Преливање (spill) – уколико у процесу упрошћавања дође до тога да ниједан преостали чвор нема ранг мањи од K , а чворови још постоје, граф није могуће обојити са K боја и долази до преливања, тј. алокација није успешна.
4. Избор (select) – чворови се узимају са врха стека (обрнутим редоследом којим су уклањани) и сваком се додељује боја (регистар) коју не користи ни један његов сусед.

Бојење графа није јединствено, различити редоследи избора слободног регистра могу дати различите (али не мање исправне) расподеле боја.

2.4 Генерисање кода

Након успешне алокације регистара приступа се последњој фази – генерисању МИПС кода, који се уписује у излазну датотеку са екстензијом `.s`. Излазни код подељен је у две секције :

- `.data` секција – служи за уписивање меморијских променљивих (попут `m1: .word 6`)

- .text секција – служи за смештање инструкција програма, чије се име преузима из декларације `_func`.

На крају се симболичке регистарске променљиве замењују додељеним физичким регистрима. Након тога можемо да упишемо резултујући садржај у излазну датотеку („output.s“) која представља резултат рада преводиоца.

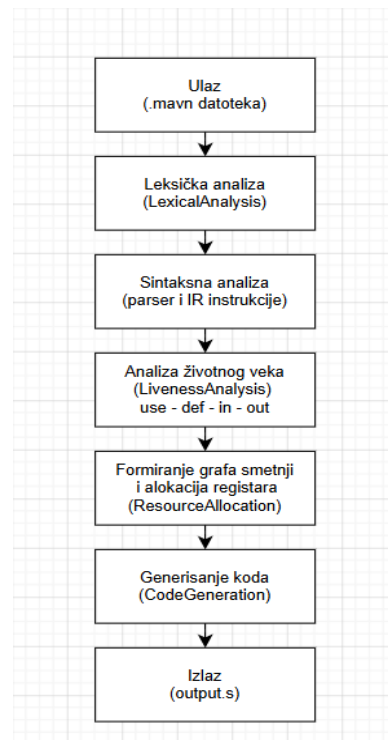
3. Концепт решења

3.1 Ток обраде програма

Решење је организовано као класичан преводилац са одвојеним фазама рада. Свака фаза добија резултат претходне фазе и припрема потребне податке за наредну фазу.

Редослед фаза је следећи :

1. Лексичка анализа
2. Синтаксна анализа
3. Анализа животног века
4. Формирање графа сметњи
5. Алокација регистара
6. Генерисање кода



Слика 2 – Ток обраде преводиоца

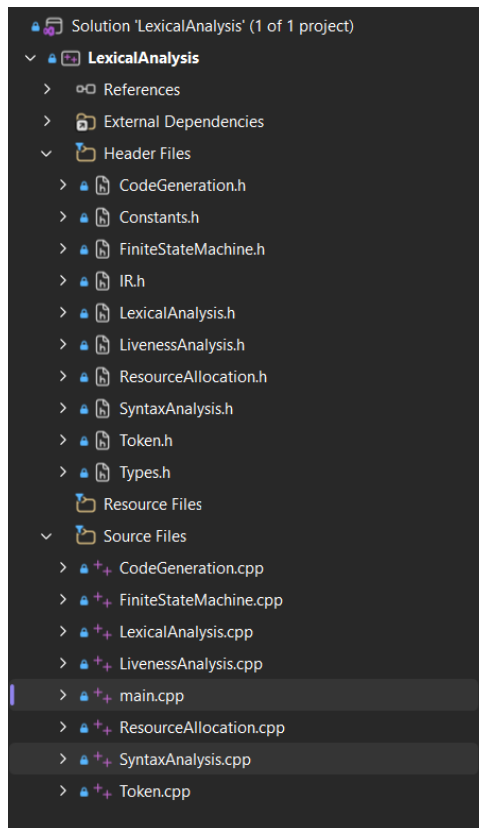
Свака фаза посебно враћа информацију о успешности извршавања. Тек када се све фазе успешно изврше, генерише се излазна датотека `output.s`. Ток обраде приказан је на слици 2.

3.2 Архитектура система

Архитектура преводиоца организована је кроз више модула. Свака фаза преводиоца реализована је у засебном модулу, смештеном у одговарајући .h односно .cpp фајл. Тиме смо раздвојили одговорности модула. Структура у оквиру Visual Studio пројекта приказана је на слици 3.

Модули пројекта :

- LexicalAnalysis, FiniteStateMachine, Token – заједно реализују лексичку анализу, преузети су из почетне структуре пројекта и проширени додатним инструкцијама и токенима
- SyntaxAnalysis – реализује синтаксну анализу и формирање међурепрезентације
- IR – садржи међурепрезентацију (заједничке класе Variable и Instruction)
- LivenessAnalysis – реализује анализу животног века (use/def/in/out)
- ResourceAllocation – обједињује формирање графа сметњи, упрошћавање, преливање и алокацију регистара
- CodeGeneration – реализује генерисање МИПС кода и упис у излазну датотеку
- main.cpp – главни програм који повезује све модуле у један ток извршавања



Слика 3 – структура пројекта

3.3 Међурепрезентација (IR)

Међурепрезентација програма заснована је на класама `Variable` и `Instruction`, дефинисаним у фајлу `IR.h`.

Класа `Variable` описује једну променљиву из МАН програма. Садржи тип променљиве (меморијска или регистарска), назив, позицију у матрици сметњи, додељени физички регистар, код меморијских променљивих и њихову вредност.

Класа `Instruction` описује једну инструкцију. Поред типа инструкције и одредишних `dst` и изворишних `src` операнда, садржи и скупове `use`, `def`, `in`, `out` потребне за анализу животног века, листе претходника и следбеника у графу тока као и лабеле и константе потребне за генерисање кода.

3.4 Процес превођења

Процес превођења почиње читавањем `.main` датотеке. Лексички анализатор дели улазни текст на токене, а синтаксни анализатор затим проверава да ли су токени написани у складу са граматиком МАН језика и истовремено кроз пролаз формира међурепрезентацију – листе променљивих и инструкција.

Над формираном међурепрезентацијом врши се анализа животног века, којом се за сваку инструкцију одређују `use`, `def`, `in`, `out` скупови. На основу `out` скупова формира се граф сметњи, а затим се поступком бојења графа свакој регистарској променљивој додељује један од 4 физичка регистра (`t0`, `t1`, `t2`, `t3`). У случају да бојење графа сметњи захтева више од 4 боје, пријављује се преливање и прекида се извршавање преводиоца.

На крају ако је све успешно, генератор кода пролази кроз листу инструкција и сваку преводи у одговарајућу МИПС инструкцију, замењујући логичке променљиве алоцираним физичким регистрима. Резултат се исписује у излазну датотеку `output.s`, која представља резултат успешног превођења.

4. Имплементација решења

4.1 Главни програм

Главни програм налази се у датотеци `main.cpp`. Његова улога је да повеже све модуле у јединствен ток извршавања. У главном програму редом се позивају лексичка анализа, синтаксна анализа, анализа животног века, алокација регистара и генерисање кода.

Свака фаза враћа информацију о успешности. Уколико лексичка или синтаксна анализа не успе, програм исписује поруку о грешци и прекида рад. Слично, уколико током алокације регистара дође до преливања, исписује се порука да додела регистара није могућа.

4.2 Модул лексичке анализе

4.2.1 Коначни аутомат

Лексичка анализа реализована је помоћу коначног аутомата стања. Аутомат пролази кроз улазни текст карактер по карактер, и на основу матрице прелаза одређује ком типу токена припада прочитана секвенца карактера. Када аутомат пређе у завршно стање, формира се одговарајући токен. На овај начин се препознају резервисане речи, идентификатори, бројеви и оператори.

4.2.2 Токени

Токен представља најмању логичку јединицу програма коју синтаксна анализа даље обрађује. У оквиру пројекта подржани су следећи типови токена : резервисане речи (T_MEM за `_mem`, T_REG за `_reg`, T_FUNC за `_func`), инструкције (T_ADD, T_SUB, T_LW, T_AND, T_OR, T_BEQ и друге), идентификатори (T_ID), регистарске и меморијске променљиве (T_R_ID, T_M_ID), бројеви (T_NUM), као и оператори (T_COMMA, T_L_PARENT, T_R_PARENT, T_COL, T_SEMI_COL).

4.2.3 Метода „Do()“

Метода “Do()” извршава комплетну лексичку анализу. Она у петљи позива функцију за добијање наредног токена све док се не дође до краја улазне датотеке (T_END_OF_FILE). Сваки препознати токен додаје се у листу токена, која се прослеђује синтаксној анализи. Уколико се наиђе на низ карактера који не одговара ни једном валидном токenu, компајлер пријављује да се десила лексичка грешка.

4.3 Модул синтаксне анализе

4.3.1 Граматика МАН језика

Синтаксна анализа заснована је на граматичи МАН језика. Граматика се састоји од терминала и нетерминала. Терминали су конкретни токени који се појављују у

програму, а нетерминали (у нашем случају Q, S, E, L) су целине које се разлажу на токене применом правила граматике.

Синтакса МАНВ језика описана је граматиком:

$Q \rightarrow S ; L$	$S \rightarrow _mem \ mid \ num$	$L \rightarrow eof$	$E \rightarrow add \ rid, \ rid, \ rid$
	$S \rightarrow _reg \ rid$	$L \rightarrow Q$	$E \rightarrow addi \ rid, \ rid, \ num$
	$S \rightarrow _func \ id$		$E \rightarrow sub \ rid, \ rid, \ rid$
	$S \rightarrow id : E$		$E \rightarrow la \ rid, \ mid$
	$S \rightarrow E$		$E \rightarrow lw \ rid, \ num(rid)$
			$E \rightarrow li \ rid, \ num$
			$E \rightarrow sw \ rid, \ num(rid)$
			$E \rightarrow b \ id$
			$E \rightarrow bltz \ rid, \ id$
			$E \rightarrow nop$

Терминални симболи МАНВ језика су:

`::, ( )`

`\_mem, \_reg, \_func, num id, rid, mid, eof, add, addi, sub, la, lw, li, sw, b, bltz, nop`

Слика 4 – граматика МАНВ језика

4.3.2 Реализација парсера

Парсер је рализован методом рекурзивног спуста (top-down), при чему свако правило граматике одговара једној методи : Q(), L(), S(), E(). Анализа почиње методом Do(), која покреће парсирање токена од почетног нетерминала.

Кључна помоћна метода је eat(TokenType t) која проверава да ли је текући токен очекиваног типа у складу са граматиком. Уколико јесте, прелази се на наредни токен, а ако није пријављује се синтаксна грешка путем методе printSyntaxError.

4.3.3 Формирање међурепрезентације

Поред провере исправности, парсер током рада формира међурепрезентацију програма. При обради декларација креирају се одговарајући објекти класе Variable и додају у листу променљивих. При обради инструкција креирају се објекти класе Instruction, при чему се одмах одређују dst и src операнди.

Подела на `dst` и `src` је кључна за наредну фазу (`dst` садржи регистре које инструкција уписује, а `src` оне које чита). Додавање готове инструкције обавља се методом `emit`.

4.4 Модул анализе животног века

Анализа животног века реализована је у модулу `LivenessAnalysis`, а покреће се методом `Do()` која редом позива три корака.

4.4.1 Одређивање `use` и `def` скупова

Метода `fillUseDef()` за сваку инструкцију попуњава скупове `use` и `def`. Пошто је парсер већ одредио `src` и `dst` операнде, овај корак их само преузима (`use` преузима `src`, а `def` преузима `dst`).

4.4.2 Формирање графа тока управљања

Метода `buildControlFlow()` за сваку инструкцију одређује скупове следбеника и претходника (`succ` и `pred`). Код класичних инструкција, следбеник је наредна инструкција у програму. Код безусловних скокова (`b`), следбеник је инструкција означена циљном лабелом. Код условних скокова (`bltz`, `beq`), постоје два следбеника : циљна инструкција и наредна инструкција. Скупови претходника изводе се из скупова следбеника.

4.4.3 Рачунање `in` и `out` скупова

Метода `calculateInOut()` итеративно рачуна `in` и `out` скупове према описаним једначинама. Поступак пролази кроз инструкције уназад (ради мањег броја итерација) и понавља се све док се скупови у две узастопне итерације не изједначе. Резултат ове методе представља основу за формирање графа сметњи.

4.5 Модул графа сметњи и алокације регистара

Овај модул је реализован у класи `ResourceAllocation`, обједињује формирање графа сметњи и комплетну алокацију регистара. Покреће се методом `Do()`, која враћа `true` уколико је алокација успешне, односно `false`, ако је дошло до преливања (`spill`). Граф

сетњи представљен је структуром `InterferenceGraph`, која садржи листу регистарских променљивих и матрицу сетњи.

4.5.1 Иницијализација и формирање графа сетњи

Метода `initVariables()` издваја регистарске променљиве и свакој додељује позицију (`pos`) – индекс у матрици сетњи. Меморијске променљиве се при томе прескачу јер се не алоцирају у регистре.

Метода `buildInterferenceGraph()` формира матрицу сетњи на основу `out` скупова из модула анализе животног века. Добијена матрица је симетрична и квадратна, што значи ако променљива *A* смета променљивој *B*, тада и *B* смета *A*.

4.5.2 Упрошћавање

Метода `doSimplification()` реализује фазу упрошћавања. У петљи се проналазе чворови чији је број суседа мањи од $K = 4$, и постављају се на стек, при чему се означавају као уклоњени из графа. Уклањањем чвора смањује се број суседа осталим чворовима, чиме се добијају нови чворови спремни за упрошћавање. Уколико у некој итерацији ниједан преостали чвор нема мање од 4 суседа, а још увек има чворова, метода враћа `false`, чиме сигнализирамо преливање.

4.5.3 Избор боја (регистара)

Метода `doSelection()` реализује фазу избора боја графа. Чворови се узимају са врха стека (редослед обрнут од оног којим су уклањани из графа сетњи). За сваки чвор прегледају се сви суседни и утврђује се које су боје (регистри) већ заузете, а затим се чвору додељује прва слободна боја (регистар). Уколико ни један регистар није слободан, метода враћа `false` и означава преливање. Када су сви чворови успешно обојени, свакој регистарској променљивој додељен је један од 4 физичка регистра.

4.6 Модул генерисања кода

Генерисање МИПС кода имплементирано је у модулу `CodeGeneration`, који као улаз прима листу променљивих, листу инструкција и име функције. Покреће се методом `generate()`, која отвара излазну датотеку и у њу уписује генерисани код подељен у секције.

4.6.1 Превођење регистара и инструкција

Најважнији корак генерисања кода јесте превођење логичких регистарских променљивих у физичке регистре. Метода `regToString()` за дату променљиву враћа одговарајући МИПС регистар (`$t0`, `$t1`, `$t2` или `$t3`) на основу регистра који јој је додељен у фази алокације.

Метода `instructionToString()` преводи једну инструкцију у њен текстуални МИПС облик. У зависности од типа инструкције, формира се одговарајући запис. При томе се користе и сачуване нумеричке константе и лабеле.

4.6.2 Формирање секција и излазне датотеке

У `.data` секцију уписују се све меморијске променљиве. Свака променљива се генерише у формату „назив: `.word` вредност“. За то је задужена метода `generateDataSection()`.

У `.text` секцију уписују се све инструкције програма преведене у МИПС облик. На почетку се генерише лабела функције. Име функције се узима из декларације `_func`, па се за `_func main` генерише `.globl main` као и `main` лабела. За то је задужена метода `generateTextSection()`. Коначан резултат је исправно преведена излазна датотека `output.s` која представља резултат рада МИПС преводиоца.

4.7 Додатне инструкције

Поред инструкција дефинисаних спецификацијом задатка, имплементирани су 3 додатне асемблерске инструкције : `and`, `or` и `beq`.

4.7.1 Инструкција `and`

Инструкција `and` припада аритметичко-логичкој групи и реализује битовску коњункцију над два регистра. По формату је облика : `and $td, $ts, $tt`, има један одредишни и два изворна регистра. У модулу међурепрезентације, одредишни регистар смешта се у `dst`, а два изворна у `src` скуп.

4.7.2 Инструкција **or**

Инструкција **or** такође припада групи аритметичко-логичких инструкција и реализује битовску дисјункцију над два регистра. Њен формат је истоветан инструкцији **and**, облика : **or \$td, \$ts, \$tt**, и такође има један одредишни и два изворна регистра.

4.7.3 Инструкција **beq**

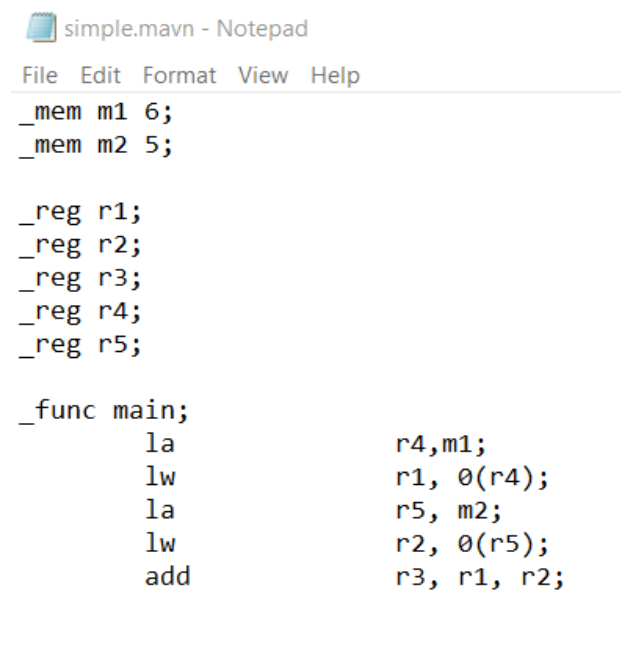
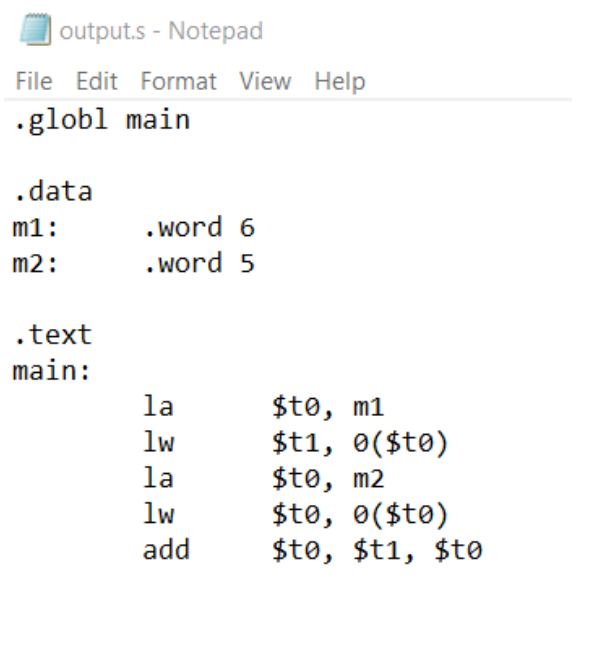
Инструкција **beq** представља условни скок и припада групи инструкција тока управљања. Инструкција врши скок на задату лабелу само уколико су вредности два регистарска операнда једнаке. За разлику од инструкција **and**, **or**, инструкција **beq** не уписује резултат у одредишни регистар, већ користи два изворишна регистра и циљну лабелу. Облик генерисане инструкције : **beq reg1, reg2, naziv_labele**.

5. Тестирање и верификација решења

Исправно функционисање преводиоца проверено је на више тест примера, изабраних тако да покрију различите аспекте рада. За сваки пример проверавано је да су резултати појединачних модула исправни тј. валидни, као и да је генерисани излазни МИПС код коректан.

5.1 Тест **simple.mavn**

Тест **simple.mavn** користи се за проверу основног превођења меморијских и регистарских променљивих, као и инструкција **la**, **lw**, **add**. Улазни програм садржи две меморијске променљиве и неколико регистарских. Видети слике 5 и 6.

 <pre> simple.mavn - Notepad File Edit Format View Help _mem m1 6; _mem m2 5; _reg r1; _reg r2; _reg r3; _reg r4; _reg r5; _func main; la r4,m1; lw r1, 0(r4); la r5, m2; lw r2, 0(r5); add r3, r1, r2; </pre> Слика 5 – simple.mavn	 <pre> outputs.s - Notepad File Edit Format View Help .globl main .data m1: .word 6 m2: .word 5 .text main: la \$t0, m1 lw \$t1, 0(\$t0) la \$t0, m2 lw \$t0, 0(\$t0) add \$t0, \$t1, \$t0 </pre> Слика 6 – simple.mavn генерисани код
--	---

Резултат превођења показује да је анализом животног века утврђено да истовремено живе највише 2 променљиве, што можемо потврдити на основу излаза (видимо само \$t0 и \$t1 регистре). Закључак : генерисани МИПС код је исправан и програм је успешно преведен.

5.2 Тест multiply.mavn

Овај пример демонстрира понашање преводиоца када број потребних регистара премашује број доступних. Програм садржи петљу у којој је у тренутку истовремено живо 5 променљивих. Будући да нам у том случају треба граф са $K = 5$ различитих боја, граф није могуће обојити са свега 4 регистра. Фаза упрошћавања се зауставља, преводилац детектује преливање и исписује поруку да додела регистара није могућа. Овај пример потврђује да преводилац исправно детектује ситуацију преливања. Видети слике 7 и 8.

multiply.mavn - Notepad

File Edit Format View Help

```
_mem m1 6;
_mem m2 5;
_mem m3 0;

_reg r1;
_reg r2;
_reg r3;
_reg r4;
_reg r5;
_reg r6;
_reg r7;
_reg r8;

_func main;
    la            r1, m1;
    lw            r2, 0(r1);
    la            r3, m2;
    lw            r4, 0(r3);
    li            r5, 1;
    li            r6, 0;
lab:
    add           r6, r6, r2;
    sub           r7, r5, r4;
    addi          r5, r5, 1;
    bltz          r7, lab;

    la            r8, m3;
    sw            r6, 0(r8);
    nop;
```

Слика 7 – multiply.mavn

```
Microsoft Visual Studio Debug Console

[5] tip=5 dst: { r6 } src: { }
[6] tip=1 dst: { r6 } src: { r6 r2 }
[7] tip=3 dst: { r7 } src: { r5 r4 }
[8] tip=2 dst: { r5 } src: { r5 }
[9] tip=8 dst: { } src: { r7 }
[10] tip=4 dst: { r8 } src: { }
[11] tip=7 dst: { } src: { r6 r8 }
[12] tip=10 dst: { } src: { }

=== Analiza zivotnog veka ===
[0] use: { } def: { r1 } succ: { 1 } pred: { } in: { } out: { r1 }
[1] use: { r1 } def: { r2 } succ: { 2 } pred: { 0 } in: { r1 } out: { r2 }
[2] use: { } def: { r3 } succ: { 3 } pred: { 1 } in: { r2 } out: { r3 r2 }
[3] use: { r3 } def: { r4 } succ: { 4 } pred: { 2 } in: { r3 r2 } out: { r2 r4 }
[4] use: { } def: { r5 } succ: { 5 } pred: { 3 } in: { r2 r4 } out: { r2 r5 r4 }
[5] use: { } def: { r6 } succ: { 6 } pred: { 4 } in: { r2 r5 r4 } out: { r6 r2 r5 r4 }
[6] use: { r6 r2 } def: { r6 } succ: { 7 } pred: { 5 9 } in: { r6 r2 r5 r4 } out: { r5 r4 r6 r2 }
[7] use: { r5 r4 } def: { r7 } succ: { 8 } pred: { 6 } in: { r5 r4 r6 r2 } out: { r5 r7 r6 r2 r4 }
[8] use: { r5 } def: { r5 } succ: { 9 } pred: { 7 } in: { r5 r7 r6 r2 r4 } out: { r7 r6 r2 r5 r4 }
[9] use: { r7 } def: { } succ: { 6 10 } pred: { 8 } in: { r7 r6 r2 r5 r4 } out: { r6 r2 r5 r4 }
[10] use: { } def: { r8 } succ: { 11 } pred: { 9 } in: { r6 } out: { r6 r8 }
[11] use: { r6 r8 } def: { } succ: { 12 } pred: { 10 } in: { r6 r8 } out: { }
[12] use: { } def: { } succ: { } pred: { 11 } in: { } out: { }

SPILL detected! Cannot allocate registers.

C:\Users\Trile\Desktop\OPPiSA\projekat\Projekat\src\Debug\LexicalAnalysis.exe (process 4800) exited with code 0 (0x0).
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the console when debugging stops.
Press any key to close this window . . .
```

Слика 8 – multiply.mavn терминал – детектовано преливање

5.3 Тест extra.mavn

Овај тест је осмишљен тако да у једном програму обухвати највећи број имплементираних инструкција. Користе се инструкције за приступ меморији la, lw, sw, све аритметичко-логичке операције add, sub, and, or, addi, и инструкција пор, над 4 регистарске променљиве и 3 меморијске локације. Анализом животног века утврђено је да су у сваком тренутку живе највише 3 променљиве, те имамо довољно регистара за алокацију у свакој инструкцији. Такође можемо приметити да се регистар t1 више пута користи за поновно адресирање чиме се постиже да се овај регистар више пута заузима и ослобађа. Искоришћени су регистри \$t0, \$t1, \$t2. Видети слике 9 и 10.

```
extra.mavn - Notepad
File Edit Format View Help
_mem m1 10;
_mem m2 20;
_mem m3 0;

_reg r1;
_reg r2;
_reg r3;
_reg r4;

_func main;
    la      r1, m1;
    lw      r2, 0(r1);
    la      r1, m2;
    lw      r3, 0(r1);
    add     r4, r2, r3;
    sub     r4, r4, r2;
    and     r4, r4, r3;
    or      r4, r4, r2;
    addi    r4, r4, 5;
    la      r1, m3;
    sw      r4, 0(r1);
    nop;
```

Слика 9 – extra.mavn

```
outputs - Notepad
File Edit Format View Help
.globl main

.data
m1:      .word 10
m2:      .word 20
m3:      .word 0

.text
main:
    la      $t1, m1
    lw      $t2, 0($t1)
    la      $t1, m2
    lw      $t1, 0($t1)
    add     $t0, $t2, $t1
    sub     $t0, $t0, $t2
    and     $t0, $t0, $t1
    or      $t0, $t0, $t2
    addi    $t0, $t0, 5
    la      $t1, m3
    sw      $t0, 0($t1)
    nop
```

Слика 10 – extra.mavn генерисани код

5.4 Тест loop.mavn

Тест jump.mavn проверава рад 3 додатне инструкције and, or, beq. Такође проверава и рад са више лабела и скокова у програму. Програм најпре учита 2 вредности из меморијских променљивих m1 и m2, а затим над њима извршава битовске операције and и or. Инструкцијом and рачуна да ли два броја имају заједничке битове постављене на 1. Уколико је резултат and операције једнак нули, инструкција beq врши скок на лабелу на којој се за резултат узима вредност добијена од операцијом. У супротном, као резултат се задржава вредност добијена операцијом and. Видети слике 11 и 12.

```
loop.mavn - Notepad
File Edit Format View Help

_mem m1 12;
_mem m2 10;
_mem m3 0;

_reg r1;
_reg r2;
_reg r3;
_reg r4;
_reg r5;
_reg r6;

_func main;
    la      r1, m1;
    lw      r2, 0(r1);

    la      r1, m2;
    lw      r3, 0(r1);

    li      r4, 0;

    and     r5, r2, r3;
    or      r6, r2, r3;

    beq     r5, r4, nema_zajednickih;

    b       upis;

nema_zajednickih:
    or      r5, r6, r4;

upis:
    la      r1, m3;
    sw      r5, 0(r1);
    nop;
```

Слика 11 – loop.mavn генерисани код

```
outputs - Notepad
File Edit Format View Help

.globl main

.data
m1:      .word 12
m2:      .word 10
m3:      .word 0

.text
main:
    la      $t0, m1
    lw      $t3, 0($t0)
    la      $t0, m2
    lw      $t0, 0($t0)
    li      $t2, 0
    and     $t1, $t3, $t0
    or      $t0, $t3, $t0
    beq     $t1, $t2, nema_zajednickih
    b       upis
nema_zajednickih:
    or      $t1, $t0, $t2
upis:
    la      $t0, m3
    sw      $t1, 0($t0)
    nop
```

Слика 12 – loop.mavn генерисани код

6. Закључак

У оквиру пројектног задатка, реализован је МАВН преводац који преводи програм написан на вишем МИПС 32-bit асемблерском језику у основни МИПС 32-bit асемблерски код. Решење обухвата комплетан ток рада преводиоца укључујући фазе : лексичке и синтаксне анализе, формирање међурепрезентације, анализе животног века променљивих, формирање графа сметњи, алокацију регистара и генерисање излазног МИПС кода. Посебан део пројекта односи се на распоређивање регистарских променљивих на ограничен број физичких регистара. На основу анализе животног века

формирали смо граф сметњи, а затим поступком бојења графа свакој променљивој доделили један од 4 доступна регистра.

Поред основних инструкција језик је проширен инструкцијама `and`, `or`, `beq`. Израдом пројекта приказана је практична примена основних концепата преводилаца.